

Quiero que construyas un MVP funcional de una aplicación web interna llamada Torre de Control.

La Torre de Control NO es un producto para vender. Es la herramienta interna de nuestra software house para administrar todas las aplicaciones SaaS que desarrollamos, sus clientes, tenants, costos, suscripciones, versiones y deployments.

CONTEXTO ACTUAL

Ya existe una aplicación llamada Mostrador, utilizada por puestos/comercios del Mercado Central.

Mostrador actualmente utiliza:

- React 19
- TypeScript
- Vite
- Tailwind
- TanStack Start
- Better Auth
- PostgreSQL
- Neon

La aplicación tiene arquitectura multi-tenant.

Cada puesto/comercio es un tenant.

La Torre de Control debe utilizar la MISMA instancia PostgreSQL/Neon existente, pero sus tablas administrativas deben estar claramente separadas de las tablas operativas de Mostrador.

NO migres la base de datos.

NO reemplaces Neon.

NO reemplaces Better Auth.

NO rehagas Mostrador.

NO rompas ninguna funcionalidad existente.

La Torre debe agregarse como una aplicación administrativa independiente.

OBJETIVO DEL MVP

Quiero tener una primera versión funcional en aproximadamente 2 días.

Priorizar funcionalidad y simplicidad sobre diseño sofisticado.

La aplicación debe permitir:

1. administrar aplicaciones;
2. administrar clientes;
3. administrar tenants;
4. administrar planes;
5. registrar suscripciones;
6. registrar ingresos;
7. registrar gastos;
8. controlar versiones;
9. registrar deployments;
10. visualizar un dashboard general.

No implementar todavía CRM completo, facturación completa, contabilidad, soporte avanzado ni automatizaciones complejas.

ARQUITECTURA

Crear un módulo administrativo separado.

Utilizar PostgreSQL/Neon existente.

Crear tablas con prefijo "saas_" para evitar conflictos:

- saas_apps
- saas_clients
- saas_tenants
- saas_plans
- saas_subscriptions
- saas_payments
- saas_expenses
- saas_versions
- saas_deployments

Si la arquitectura existente recomienda otra convención equivalente, mantener la idea de separación lógica.

MODELO DE DATOS

saas_apps

Campos mínimos:

- id
- name

- slug
- description
- status
- repository_url
- production_url
- current_version
- created_at
- updated_at

Ejemplo:

Mostrador / mostrador

saas_clients

Campos:

- id
- name
- business_name
- contact_name
- phone
- email
- notes
- status
- created_at
- updated_at

saas_tenants

Campos:

- id
- app_id
- client_id
- name
- slug
- status
- plan_id
- installed_version
- last_activity_at
- created_at
- updated_at

Un tenant pertenece a una aplicación y a un cliente.

No crear un tenant por dispositivo.

saas_plans

Campos:

- id
- name
- description
- monthly_price
- setup_price
- status
- created_at
- updated_at

saas_subscriptions

Campos:

- id
- client_id
- tenant_id
- plan_id
- monthly_price
- status
- start_date
- next_payment_date
- created_at
- updated_at

saas_payments

Campos:

- id
- client_id
- subscription_id
- amount
- payment_date
- concept

- status
- notes
- created_at

saas_expenses

Campos:

- id
- app_id nullable
- concept
- provider
- amount
- frequency
- expense_date
- notes
- created_at

Debe permitir registrar costos como:

- hosting
- base de datos
- dominios
- servicios de IA
- herramientas de desarrollo
- APIs
- otros servicios

saas_versions

Campos:

- id
- app_id
- version
- platform
- release_date
- changelog
- download_url nullable
- status
- created_at

Plataformas iniciales:

- web
- android
- windows

saas_deployments

Campos:

- id
- app_id
- tenant_id nullable
- version_id
- platform
- status
- deployed_at
- notes

Estados:

- pending
- deployed
- failed
- rolled_back

DASHBOARD

Crear una pantalla inicial clara y simple.

Mostrar:

Indicadores

- aplicaciones activas
- clientes activos
- tenants activos
- ingresos mensuales
- gastos mensuales
- margen mensual

Aplicaciones

Mostrar:

- nombre

- tenants
- versión actual
- estado

Alertas

Detectar automáticamente:

- tenants con versión desactualizada;
- pagos vencidos;
- aplicaciones sin versión;
- deployments fallidos.

TENANTS

Crear una vista de tenants.

Columnas:

- Tenant
- Aplicación
- Cliente
- Plan
- Estado
- Versión instalada
- Última actividad
- Acciones

Acciones:

- ver
- editar
- desactivar

CONTROL DE VERSIONES

Crear pantalla:

Aplicación → versiones.

Ejemplo:

Mostrador

Versión disponible:

1.0.5

Tenants:

Frutas Román — 1.0.5 ✓

Puesto 02 — 1.0.5 ✓

Puesto 03 — 1.0.4 ⚠

Puesto 04 — 1.0.3 ⚠

La Torre debe poder identificar qué tenants están atrasados.

IMPORTANTE:

En este MVP NO implementar todavía actualización automática de Android/Windows.

Solamente preparar correctamente la arquitectura para incorporarla después mediante deployments/GitHub Actions.

SEGURIDAD

La Torre es una aplicación administrativa interna.

Implementar autenticación.

Solo usuarios autorizados pueden acceder.

No exponer públicamente endpoints administrativos sin autenticación.

No permitir que un tenant normal de Mostrador acceda a la Torre.

La separación entre:

ADMINISTRACIÓN DEL SaaS

y

DATOS OPERATIVOS DE LOS TENANTS

debe ser explícita.

UI

Utilizar el stack existente siempre que sea razonable:

React
TypeScript
Vite
Tailwind

Diseño:

- sidebar
- dashboard
- tablas
- filtros
- formularios simples
- badges de estado
- modales solamente cuando aporten valor

No perder tiempo en animaciones.

La aplicación debe sentirse como una consola administrativa profesional.

Menú:

Dashboard

Aplicaciones
Clientes
Tenants
Planes
Suscripciones
Finanzas
Versiones
Deployments

Configuración

REGLA FUNDAMENTAL

NO reconstruyas Mostrador.

NO cambies su arquitectura innecesariamente.

NO migres Neon.

NO introduzcas Supabase.

NO agregues dependencias innecesarias.

Primero inspecciona la arquitectura existente y reutiliza:

- conexión PostgreSQL
- autenticación
- componentes UI
- utilidades
- servicios
- convenciones de código

siempre que sea seguro hacerlo.

PLAN DE IMPLEMENTACIÓN

Trabaja en este orden:

FASE 1

Inspeccionar proyecto actual.

Identificar:

- estructura
- conexión Neon
- ORM/query layer
- autenticación
- rutas
- componentes
- variables de entorno
- sistema de migraciones

No modificar código existente hasta entenderlo.

FASE 2

Crear las tablas "saas_*".

Crear migración reversible.

Verificar que las tablas existentes de Mostrador continúan funcionando.

FASE 3

Crear servicios CRUD:

Apps
Clients
Tenants
Plans
Subscriptions
Payments
Expenses
Versions
Deployments

FASE 4

Crear interfaz Torre.

FASE 5

Crear Dashboard con métricas calculadas desde PostgreSQL.

FASE 6

Crear datos de prueba.

Crear al menos:

- 1 aplicación Mostrador
- 1 cliente
- 1 tenant
- 1 plan
- 1 suscripción
- 1 pago
- 2 gastos
- 2 versiones
- 1 deployment

FASE 7

Ejecutar build y tests.

Corregir errores.

No declarar terminado hasta que:

- compile;
- arranque;
- autentique;
- consulte Neon;
- cree un cliente;
- cree un tenant;

- registre una suscripción;
- registre un gasto;
- registre una versión;
- muestre correctamente el dashboard.

FUTURO

Diseñar el MVP de manera que posteriormente podamos incorporar:

1. GitHub Actions.
2. Deployments automáticos.
3. Publicación de versiones Android.
4. Publicación de versiones Windows.
5. Descarga/actualización automática de clientes.
6. Health checks.
7. Logs.
8. Monitoreo.
9. Alertas.
10. Gestión de soporte.
11. Métricas de uso.
12. Control centralizado de actualizaciones de todos los tenants.

Pero NO implementar todo eso ahora.

El objetivo inmediato es:

Tener una Torre de Control funcional en 2 días que permita administrar la software house desde una sola pantalla.

Antes de escribir código, inspecciona el proyecto existente y dame un diagnóstico corto de:

1. arquitectura actual;
2. cómo accederás a Neon;
3. dónde integrarás la Torre;
4. tablas que crearás;
5. archivos que modificarás;
6. riesgos de afectar Mostrador.

Después de ese diagnóstico, comienza la implementación.